

Stage 1.4.2.5.1 — Basic OCR

Building the Fundamental OCR Ingestion Pipeline for Scanned PDFs in RAG

Absolutely. Let's continue exactly from the saved checkpoint.

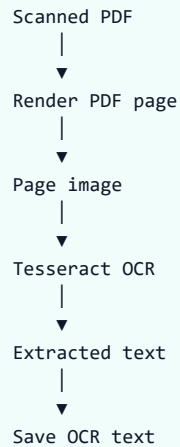
Today we are going to build the simplest complete OCR pipeline for the scanned PDF we have already been using.

Our goal is not yet to build a production OCR system. The goal is to understand exactly what happens when a RAG system encounters a scanned PDF.

1. What we are going to accomplish

By the end of this stage, you'll have implemented:

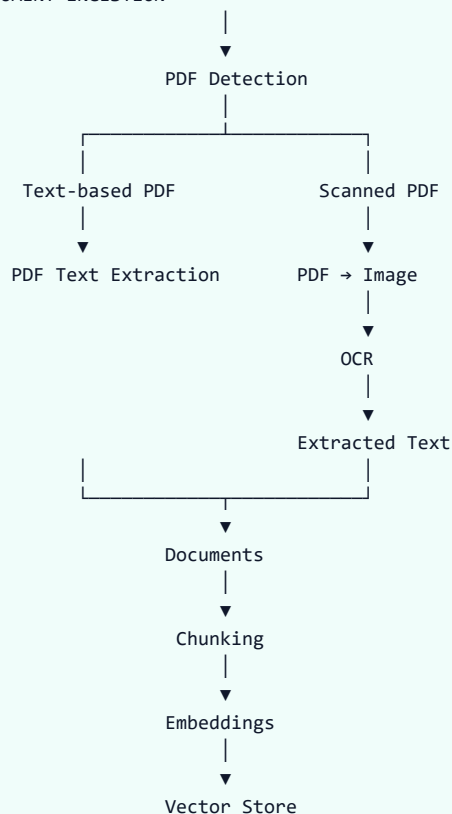
PIPELINE SEQUENCE



And we'll understand where this fits into RAG:

INGESTION ARCHITECTURE

DOCUMENT INGESTION



This is an important milestone because you're now moving from "detecting a scanned PDF" to actually processing it.

2. Before coding — understand the two-step process

There is one important concept we clarified previously.

You don't directly give a PDF to basic Tesseract OCR.

Instead:

PROCESS FLOW

PDF
↓
Page
↓
Image
↓
OCR

Why?

Because Tesseract fundamentally works with images.

For example:

MULTI-PAGE BREAKDOWN

```

scanned_document.pdf
├── Page 1 → image
├── Page 2 → image
└── Page 3 → image

```

Then:

```

Page 1 image → Tesseract → text
Page 2 image → Tesseract → text
Page 3 image → Tesseract → text

```

For today's first experiment, we'll deliberately process one page.

3. Our environment

You are using:

- Windows 10
- VS Code
- Jupyter Notebook
- uv
- your rag-learning project

We'll continue using that environment rather than creating another project.

4. Step 1 — Install Tesseract OCR

There are two separate things involved:

TWO COMPONENTS

Tesseract OCR

+

pytesseract

Tesseract

The actual OCR engine.

pytesseract

Python wrapper that allows Python to communicate with Tesseract.

So:

COMMUNICATION FLOW

Python Notebook

↓

pytesseract

↓

Tesseract OCR Engine

↓

Image → Text

For Windows, use the Tesseract Windows distribution from UB Mannheim:

[Tesseract Windows installation information](#)

Install it with the English language data (eng).

A common installation path is:

PATH

C:\Program Files\Tesseract-OCR\tesseract.exe

5. Step 2 — Verify Tesseract

Open a new PowerShell terminal.

Run:

POWERSHELL

```
tesseract --version
```

You should see something similar to:

OUTPUT

```
tesseract 5.x.x  
leptonica-...
```

Then:

POWERSHELL

```
tesseract --list-langs
```

You should see at least:

OUTPUT

```
eng  
osd
```

Why are we doing this outside Python?

Because we want to separate:

Is Tesseract installed correctly?

from:

Can Python communicate with Tesseract?

This is a good troubleshooting habit for production systems too.

6. Step 3 — Install pytesseract

From your existing rag-learning project root:

BASH

```
uv add pytesseract
```

We don't install Tesseract itself using uv.

Remember:

DEPENDENCY RESPONSIBILITY

```
uv  
├──  
└── Python dependencies  
    └── pytesseract
```

```
Windows  
├──  
└── Tesseract OCR executable
```

7. Step 4 — Verify from the Notebook

Open your existing Stage 1.4.2 notebook.

Create a cell:

PYTHON

```
import pytesseract

print(pytesseract.get_tesseract_version())
```

If it works, you'll see your Tesseract version.
Then:

PYTHON

```
print(pytesseract.get_languages())
```

You should see eng.

8. What if Python cannot find Tesseract?

You may encounter: `TesseractNotFoundError`

Don't worry if this happens.
It simply means:

LOOKUP FAILURE

```
Python
↓
pytesseract
↓
Cannot locate tesseract.exe
```

You can explicitly tell pytesseract where Tesseract is installed:

PYTHON

```
pytesseract.pytesseract.tesseract_cmd = (
    r"C:\Program Files\Tesseract-OCR\tesseract.exe"
)
```

Then:

PYTHON

```
print(pytesseract.get_tesseract_version())
```

If that works, we're ready.

9. Step 5 — Locate our scanned PDF

Now we come to the part we corrected in our previous conversation.

We do not already have scanned_page.png.

We have our scanned PDF.

So first identify its path.

For example:

PYTHON

```
pdf_path = "path/to/your/scanned.pdf"
```

Don't blindly use that exact path.

Use the actual path of the scanned PDF you've been using in your previous notebook.

You can verify:

PYTHON

```
from pathlib import Path

pdf_path = Path("path/to/your/scanned.pdf")

print(pdf_path.exists())
print(pdf_path)
```

We want: True

10. Step 6 — Render the PDF page as an image

For this part, we'll use PyMuPDF, which you've already encountered during our PDF-loader comparison. Conceptually:

RENDERING PIPELINE



Import it:

PYTHON

```
import fitz
```

Open the PDF:

PYTHON

```
pdf = fitz.open(pdf_path)

print("Number of pages:", len(pdf))
```

Now let's take the first page:

PYTHON

```
page = pdf[0]
```

Render it:

PYTHON

```
pix = page.get_pixmap()
```

Save it:

PYTHON

```
image_path = "scanned_page.png"

pix.save(image_path)

print(f"Saved: {image_path}")
```

Now we have actually created: scanned_page.png
This is the intermediate artifact we were talking about earlier.

11. Inspect the generated image

Before OCR, let's see what we created.

PYTHON

```
from PIL import Image
from IPython.display import display

image = Image.open(image_path)

print("Image size:", image.size)
print("Image mode:", image.mode)

display(image)
```

This step is extremely important.

We want to visually confirm:

"Yes, this image really represents the scanned PDF page."

12. Step 7 — Run our first OCR

Now comes the actual OCR.

PYTHON

```
ocr_text = pytesseract.image_to_string(
    image,
    lang="eng"
)
```

Print it:

PYTHON

```
print(ocr_text)
```

□ This is your first real OCR operation in our RAG learning journey.

We have gone from:

EXTRACTION TRANSFORMATION

```
Scanned PDF
  ↓
Image
  ↓
OCR
  ↓
Text
```

13. Let's inspect the OCR output properly

Instead of only looking at the text, let's examine it.

PYTHON

```
print("Characters extracted:", len(ocr_text))
print("Lines extracted:", len(ocr_text.splitlines()))

print("=" * 80)
print(ocr_text)
print("=" * 80)
```

Now compare the output with the original scanned page.

Ask yourself:

- Text accuracy: Are the words correct?
- Formatting: Are paragraphs preserved?
- Numbers: Are numbers correctly recognized?
- Symbols: Are special characters correct?
- Headings: Are headings detected correctly?

This comparison is going to become very important when we reach: Stage 1.4.2.5.3 — OCR Quality Evaluation

14. Step 8 — Save the OCR result

Let's save our first OCR output:

PYTHON

```
ocr_output_path = "scanned_page_ocr.txt"

with open(
    ocr_output_path,
    "w",
    encoding="utf-8"
) as f:
    f.write(ocr_text)

print(f"OCR output saved to: {ocr_output_path}")
```

Now your experiment has produced:

PROJECT STRUCTURE

```
your-project/
|
├── scanned.pdf
|
├── scanned_page.png
|
└── scanned_page_ocr.txt
```

The exact directory structure can vary depending on where your notebook and PDF are located.

15. What have we actually learned?

This looks simple, but conceptually it is extremely important.

We started with:

PDF

But the PDF didn't have a useful text layer.

Therefore:

CONCEPTUAL SUMMARY


```
PDF
↓
Page rendering
↓
Image

Then:

Image
↓
OCR
↓
Text
```

Therefore: OCR is a bridge between image-based documents and text-based RAG pipelines.
That's one of the most important concepts in this entire stage.

16. What Basic OCR does NOT solve

Suppose our scanned PDF contains:

VISUAL TABLE INPUT

Employee Report		
Name	Department	Salary
Ravi	Data	100000
Suresh	IT	120000

Basic OCR may give us something like:

FLATTENED OCR RESULT

Employee Report

Name Department Salary
Ravi Data 100000
Suresh IT 120000

But we may lose the actual table structure.

Likewise, if the PDF contains:

- tables
- diagrams
- charts
- mathematical formulas
- multiple columns
- headers/footers
- images
- complex layouts

basic OCR isn't enough.

And this connects directly to the question you asked in our previous conversation about tables, column charts, architecture diagrams, and mathematical formulas.

That's exactly why we are progressing through the OCR stages rather than stopping at basic OCR.

17. Where we are in the bigger RAG journey

Our current journey looks like this:

CURRICULUM ROADMAP

```
Stage 1 – Data Ingestion
├── Text files
├── PDF files
├── PDF loader comparison
└── Stage 1.4.2 – OCR & Complex PDF Ingestion
    ├── 1.4.2.1 – Text PDF vs Scanned PDF
    ├── 1.4.2.2 – Detecting Scanned Pages
    ├── ...
    └── 1.4.2.5 – OCR Approach
        ├── 1.4.2.5.1 – Basic OCR ← WE ARE HERE
        ├── 1.4.2.5.2 – OCR Preprocessing
        ├── 1.4.2.5.3 – OCR Quality Evaluation
        ├── 1.4.2.5.4 – OCR + PDF Page Pipeline
        └── 1.4.2.5.5 – OCR → LangChain Documents
```

One important learning rule for this stage

Don't move to preprocessing yet.

First run the basic OCR and look carefully at what Tesseract produces from our actual scanned page.

That gives us our baseline OCR result.

Then Stage 1.4.2.5.2 becomes much more meaningful because we'll be able to say:

"Here is what basic OCR gave us. Now let's improve the image before OCR and see whether the result gets better."

Your immediate notebook sequence

Run these cells in order:

NOTEBOOK CELLS EXECUTION

```
# 1. Verify Tesseract
import pytesseract
print(pytesseract.get_tesseract_version())

# 2. Open scanned PDF
import fitz

pdf = fitz.open(pdf_path)
print("Pages:", len(pdf))

# 3. Render first page
page = pdf[0]
pix = page.get_pixmap()

image_path = "scanned_page.png"
pix.save(image_path)

print(image_path)

# 4. Inspect image
from PIL import Image
from IPython.display import display

image = Image.open(image_path)

print(image.size)
display(image)

# 5. Basic OCR
ocr_text = pytesseract.image_to_string(
    image,
    lang="eng"
)

print(ocr_text)

# 6. Save OCR output
with open("scanned_page_ocr.txt", "w", encoding="utf-8") as f:
    f.write(ocr_text)

print("OCR completed.")
```

Run these against your actual scanned PDF. Once you see the OCR output, we can analyze your actual result together before moving to the next sub-stage.